PDS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 08 -

NEW RELIC - TRACES E LOGS

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SUMÁRIO

O QUE VEM POR AI? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 24
REFERÊNCIAS 25

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VEM POR AI?

Esta aula aprofundará dois pilares centrais da observabilidade em sistemas distribuídos modernos: Distributed Tracing e Log Management, utilizando a plataforma New Relic como referência tecnológica. O conteúdo explorará como requisições são rastreadas de ponta a ponta em arquiteturas de microsserviços, bem como a forma como eventos de log são coletados, estruturados, correlacionados e analisados em conjunto com traces e métricas.

Serão apresentados os fundamentos do rastreamento distribuído, incluindo a modelagem de traces e spans, estratégias de amostragem (head-based e tail-based), propagação de contexto baseada no padrão W3C Trace Context e mecanismos de visualização e análise de traces na interface do New Relic. Em complemento, a aula abordará o gerenciamento de logs em larga escala, destacando técnicas de coleta, parsing, detecção automática de padrões, enriquecimento de contexto e controle de ingestão.

A integração entre traces, logs e métricas, dentro do modelo MELT, será tratada como elemento central para viabilizar diagnósticos precisos, redução de tempo de resposta a incidentes e tomada de decisão orientada a dados em ambientes cloud-native e orientados a eventos.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

As atividades práticas desta aula se concentrarão na instrumentação de aplicações, na configuração de distributed tracing e no gerenciamento avançado de logs, explorando cenários realistas de produção.

O hands on contemplará a instrumentação automática e manual de traces distribuídos, a configuração de Infinite Tracing com amostragem baseada em cauda (tail-based sampling) e a propagação explícita de contexto em fluxos assíncronos e baseados em filas. Também serão realizadas análises avançadas de traces e spans utilizando NRQL, permitindo a identificação de gargalos de latência e erros.

No eixo de logs, as atividades incluirão a configuração de Logs in Context, a coleta de logs via Fluent Bit em ambientes conteinerizados, a criação de regras de parsing com Grok, a análise de log patterns baseada em Machine Learning e a aplicação de drop rules para otimização de ingestão. O conjunto de práticas demonstrará a correlação efetiva entre métricas, traces e logs em dashboards unificados de observabilidade

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SAIBA MAIS

Distributed Tracing

Como Funciona o Distributed Tracing

O Rastreamento Distribuído (Distributed Tracing) é uma metodologia essencial para a observabilidade de sistemas modernos, especialmente em arquiteturas de microsserviços, onde uma única requisição pode atravessar dezenas ou centenas de serviços independentes. Ele permite que desenvolvedores(as) e equipes de operações visualizem o fluxo completo de uma requisição, de ponta a ponta, através de todos os serviços que ela percorre.

No New Relic, o rastreamento distribuído é implementado através da coleta e correlação de **spans**, que representam unidades de trabalho individuais dentro de um serviço, como uma chamada de banco de dados, uma requisição HTTP externa ou a execução de um método específico.

A estrutura de um trace distribuído no New Relic é análoga a uma árvore. O primeiro serviço que recebe a requisição é considerado o **root** do trace. Cada operação subsequente dentro desse serviço, ou em serviços downstream, gera spans "filhos" que são ligados a um span "pai", formando uma hierarquia que representa o fluxo da requisição. Essa estrutura hierárquica é fundamental para entender as dependências e a latência em cada etapa do processo.

Os spans são classificados em diferentes tipos para fornecer um contexto mais detalhado sobre a operação que representam. Dentro de um processo (um serviço ou uma função), os spans podem ser categorizados como:

- **Entry span**: o primeiro span em um processo, representando o ponto de entrada da requisição naquele serviço.
- **Exit span**: um span que representa uma chamada para fora do processo atual, seja para outro serviço, um banco de dados ou um sistema de mensageria.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **In-process span**: um span que representa uma operação interna ao processo, como a execução de um método ou função, que não é um ponto de entrada nem de saída.

Além disso, os spans que representam chamadas a dependências externas são classificados como **client spans**, que por sua vez se dividem em:

- **Datastore spans**: representam chamadas a bancos de dados e são identificados por atributos com o prefixo db., como db.statement.
- **External spans**: representam chamadas a outros serviços via HTTP ou outros protocolos de rede e são identificados por atributos com o prefixo http., como http.url.

A duração total de um trace é calculada desde o início do primeiro span (no serviço root) até a conclusão do último span. Essa métrica, juntamente com a visualização da árvore de spans, permite identificar gargalos de performance, entender como a latência se propaga através dos serviços e diagnosticar erros com precisão. Todos os dados de traces e spans são armazenados no New Relic Database (NRDB) e podem ser consultados utilizando a New Relic Query Language (NRQL), permitindo análises aprofundadas e a criação de dashboards customizados.

Estratégias de Amostragem e Infinite Tracing

A coleta de todos os traces de um sistema em produção pode gerar um volume de dados massivo, resultando em altos custos de ingestão e armazenamento, além de sobrecarga de performance. Para gerenciar esse volume, o New Relic emprega diferentes estratégias de amostragem (sampling). A amostragem é o processo de selecionar um subconjunto representativo de traces para análise, em vez de capturar 100% deles.

Por padrão, os agentes APM do New Relic utilizam uma técnica chamada head-based sampling. Nessa abordagem, a decisão de receber uma amostra ou não um trace é tomada no início do seu ciclo de vida, no primeiro serviço que a requisição atinge (o "head"). O agente APM utiliza um algoritmo de adaptive sampling, que ajusta dinamicamente a taxa de amostragem com base no throughput de requisições do serviço.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O objetivo é capturar um número estatisticamente significativo de traces (por padrão, até 10 traces por minuto por agente) sem sobrecarregar o sistema ou a plataforma New Relic. A principal vantagem do head-based sampling é a sua eficiência e baixo overhead, mas a sua desvantagem é que traces interessantes, como aqueles que contêm erros ou alta latência, podem ser descartados se não forem selecionados na decisão inicial de amostragem.

Para superar as limitações do head-based sampling, o New Relic oferece uma solução mais poderosa chamada Infinite Tracing. Esta é uma abordagem de tail-based sampling, onde a decisão de reter um trace é tomada no final do seu ciclo de vida, após todos os spans terem sido coletados.

Com o Infinite Tracing, 100% dos spans de todos os serviços instrumentados são enviados para um componente chamado Trace Observer, que é um serviço hospedado na nuvem da New Relic (no New Relic Edge). O Trace Observer armazena temporariamente todos os spans de um trace e, somente após o trace ser concluído, aplica regras de amostragem para decidir quais traces devem ser persistidos para análise a longo prazo.

O Infinite Tracing permite a criação de regras de amostragem muito mais inteligentes e contextuais. Por exemplo, é possível configurar o Trace Observer para reter todos os traces que:

- Contenham erros;
- Excedam um determinado limiar de duração (latência);
- Correspondam a um determinado endpoint ou cliente.

Essa abordagem garante que os traces mais importantes para a solução de problemas e análise de performance nunca sejam perdidos. No entanto, como todos os spans são enviados para a New Relic, o Infinite Tracing pode resultar em um aumento nos custos de egress de dados da sua infraestrutura para a nuvem da New Relic. A escolha entre head-based e tail-based sampling depende, portanto, de um balanço entre a necessidade de visibilidade completa e os custos associados.

Propagação de Contexto de Trace

Para que o rastreamento distribuído funcione, é crucial que a identidade de um trace seja mantida à medida que a requisição flui entre diferentes serviços, processos

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

e até mesmo data centers. Esse processo é chamado de propagação de contexto de trace. O contexto de trace é um conjunto de metadados que viaja com a requisição, permitindo que os agentes APM do New Relic em diferentes serviços associem seus respectivos spans ao mesmo trace.

O New Relic suporta e recomenda o padrão W3C Trace Context, uma especificação aberta que padroniza a forma como o contexto de trace é propagado através de cabeçalhos HTTP. A adoção de um padrão aberto como o W3C Trace Context é fundamental para a interoperabilidade, permitindo que traces sejam correlacionados mesmo em ambientes poliglotas e com ferramentas de observabilidade de diferentes fornecedores.

Quando o rastreamento distribuído está habilitado, o agente APM do New Relic injeta automaticamente os seguintes cabeçalhos HTTP nas requisições de saída:

- traceparent: este é o cabeçalho principal e obrigatório do padrão W3C. Ele contém a versão do padrão, o traceId (o identificador único de todo o trace), o parentId (o ID do span que está fazendo a chamada de saída) e flags de amostragem.
- tracestate: este cabeçalho opcional carrega informações específicas do fornecedor (vendor-specific) em um formato de lista de pares chave-valor. O New Relic utiliza o tracestate para propagar informações adicionais, como o ID da conta, o ID da aplicação e a decisão de amostragem.

Além dos cabeçalhos W3C, o agente New Relic também pode enviar um cabeçalho proprietário, newrelic, para garantir a retrocompatibilidade com agentes mais antigos que ainda não suportam o padrão W3C Trace Context.

O fluxo de propagação de contexto funciona da seguinte maneira: quando um Serviço A faz uma chamada para um Serviço B, o agente APM no Serviço A gera um exit span e injeta os cabeçalhos traceparent e tracestate na requisição HTTP. O traceparent conterá o traceId do trace atual e o ID do exit span como parentId. Quando o Serviço B recebe a requisição, o agente APM no Serviço B extrai os cabeçalhos, reconhece que a requisição faz parte de um trace existente e cria um entry span com o mesmo traceId. O parentId deste entry span será o ID do exit span do Serviço A, conectando assim os dois serviços dentro do mesmo trace. Esse processo se repete para cada salto na cadeia de chamadas, construindo a árvore de spans completa.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Essa propagação de contexto é o que permite ao New Relic reconstruir a jornada completa de uma requisição, mesmo que ela atravesse múltiplos serviços, filas de mensagens e outros componentes de uma arquitetura distribuída.

Visualizar e Analisar Traces

A plataforma New Relic oferece uma interface de usuário rica e interativa para visualizar e analisar os dados de rastreamento distribuído, permitindo que as equipes identifiquem rapidamente a causa raiz de problemas de performance e erros. A principal tela para análise de um trace específico é a Trace Details UI, que fornece múltiplas visualizações e ferramentas para dissecar a jornada de uma requisição.

Ao abrir um trace, o usuário é apresentado a um resumo que inclui a duração total, o número de spans, o número de serviços envolvidos e a presença de erros. Abaixo do resumo, existem três visualizações principais para analisar os spans do trace:

Timeline View: esta visualização exibe todos os spans do trace em uma linha do tempo densa e compacta. É particularmente útil para obter uma visão geral rápida de como a latência e os erros se propagam através dos serviços. Spans com erros são destacados em vermelho, facilitando a identificação de problemas.

Latency View: esta visualização foca na contribuição de latência de cada entidade (serviço, banco de dados, etc.) para a duração total do trace. Ela mostra um gráfico que detalha a latência total que cada entidade adiciona e quantos erros cada uma gera. É uma forma eficaz de identificar qual serviço ou dependência é o principal gargalo de performance em uma transação.

Waterfall View: esta é a visualização mais detalhada e talvez a mais poderosa. Ela apresenta os spans em um diagrama de cascata (waterfall), mostrando a sequência de operações, a duração de cada uma e as relações de pai e filho entre elas. A Waterfall View oferece vários controles interativos:

- Expandir/Colapsar Spans: permite detalhar ou ocultar spans aninhados para focar em áreas específicas do trace.
- Modo Standard/Manual: alterna entre uma visualização padrão e uma que permite o controle manual da expansão dos spans.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- Focus on: permite destacar spans que atendem a certos critérios, como aqueles com erros, anomalias de duração ou pertencentes a um serviço específico. Esse recurso é extremamente útil para isolar rapidamente o span problemático em traces muito grandes e complexos.

Além dessas visualizações, a UI de Trace Details permite navegar diretamente do span para a entidade correspondente (por exemplo, para a página de APM do serviço que gerou o span) para uma investigação mais aprofundada. Também é possível visualizar os logs associados a um span específico (se o Logs in Context estiver configurado), fornecendo um contexto ainda mais rico para a solução de problemas.

Finalmente, todos os dados de trace podem ser consultados via NRQL, permitindo a criação de dashboards customizados, alertas sobre a duração ou a taxa de erro de traces, e análises de tendências de performance ao longo do tempo.

Log Management

Conceito e Benefícios

O gerenciamento de logs (Log Management) é um pilar fundamental da observabilidade moderna. Logs são registros de eventos que ocorrem ao longo do tempo em um sistema de software, capturando desde mensagens de depuração detalhadas até erros críticos e transações de negócios. A New Relic aborda o gerenciamento de logs com a filosofia de que logs são uma forma de telemetria de alto volume e, individualmente, de baixo valor. O verdadeiro poder dos logs é desbloqueado quando eles são agregados, correlacionados e analisados em conjunto com outras formas de telemetria, como métricas, eventos e traces (o framework MELT).

A centralização dos logs na plataforma New Relic oferece uma série de benefícios estratégicos para as equipes de desenvolvimento e operações:

Redução do Tempo Médio de Resolução (MTTR): ao ter todos os logs de aplicações, infraestrutura e serviços em um único local, correlacionados com dados de performance, as equipes podem diagnosticar a causa raiz de falhas de forma muito mais rápida. Em vez de acessar dezenas de servidores ou ferramentas diferentes, os

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

desenvolvedores podem ver os logs exatos associados a um erro de transação ou a um pico de latência com um único clique.

Visibilidade Unificada: a plataforma New Relic permite a correlação automática de logs com eventos de APM (Application Performance Monitoring), infraestrutura e experiência do usuário final (Browser e Mobile). Isso significa que é possível navegar de um erro em um trace distribuído diretamente para as linhas de log que foram escritas durante a execução daquele span, fornecendo um contexto profundo e imediato para a investigação.

Análise Proativa e Detecção de Anomalias: com todos os logs em um só lugar, é possível aplicar algoritmos de Machine Learning para detectar padrões (Log Patterns) e identificar anomalias. A plataforma pode, por exemplo, alertar sobre um aumento súbito em um tipo específico de mensagem de erro que, embora não seja crítico individualmente, pode indicar um problema sistêmico emergente.

Criação de Dashboards e Alertas: os dados de log podem ser consultados via NRQL para criar dashboards que visualizam tendências, taxas de erro e outros KPIs (Key Performance Indicators) derivados dos logs. Além disso, é possível configurar alertas precisos com base no conteúdo ou na frequência de mensagens de log, permitindo que as equipes respondam a problemas antes que eles afetem os usuários.

Em resumo, o gerenciamento de logs na New Relic transforma os logs de um recurso reativo, usado apenas após a ocorrência de um incidente, em uma fonte proativa de insights sobre a saúde e o comportamento do sistema.

Coleta de Logs e Log Forwarders

A eficácia do gerenciamento de logs depende fundamentalmente da capacidade de coletar logs de todas as fontes relevantes de forma confiável e eficiente. A plataforma New Relic oferece uma vasta gama de opções para o encaminhamento de logs (log forwarding), garantindo a compatibilidade com praticamente qualquer ambiente, desde servidores on-premise até as mais complexas arquiteturas de nuvem e contêineres.

A escolha do método de coleta depende da origem dos logs e da infraestrutura existente. As principais opções são:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Agente de Infraestrutura da New Relic: esta é frequentemente a maneira mais fácil e recomendada de encaminhar logs de arquivos em servidores físicos ou virtuais. O agente de infraestrutura pode ser configurado para monitorar (tail) arquivos de log específicos e encaminhá-los para a New Relic. Uma vantagem crucial deste método é que o agente enriquece automaticamente os logs com metadados da entidade (como entity.name, hostname, etc.), o que habilita a funcionalidade de Logs in Context.

Log Forwarders de Código Aberto: para equipes que já utilizam forwarders de log padrão da indústria, o New Relic oferece plugins e integrações para:

- Fluent Bit: um processador e encaminhador de logs leve e de alta performance, ideal para ambientes de contêineres.
- Fluentd: um coletor de dados de código aberto popular, com uma vasta gama de plugins.
- Logstash: componente do Elastic Stack (ELK), pode ser configurado para enviar logs para a New Relic como um de seus outputs.

Integrações com Provedores de Nuvem: a New Relic possui integrações diretas com os principais provedores de nuvem para coletar logs de seus serviços gerenciados:

- AWS: integração com Amazon CloudWatch e Amazon Kinesis Firehose para coletar logs de serviços como Lambda, S3, RDS, etc.
- Microsoft Azure: encaminhamento de logs do Azure Monitor e do Azure Event Hubs.
- Google Cloud Platform (GCP): coleta de logs através do Google Cloud Logging (Stackdriver).

Ambientes de Contêineres e Kubernetes: o plugin de Kubernetes da New Relic pode ser configurado para coletar logs de todos os pods em um cluster, geralmente monitorando o diretório /var/log/containers/*.log nos nós do cluster.

APM Logs in Context: os próprios agentes APM do New Relic podem ser configurados para encaminhar logs diretamente da aplicação. Esta opção é conveniente, mas a abordagem recomendada é usar um agente de infraestrutura ou

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

outro forwarder para separar as preocupações de coleta de logs da instrumentação da aplicação.

Log API e TCP Endpoint: em cenários onde as opções acima não são viáveis, é possível enviar logs diretamente para a New Relic via uma API HTTP (Log API) ou através de um endpoint TCP genérico, que pode receber logs em formato syslog ou outros formatos de texto simples.

Essa flexibilidade garante que, independentemente da arquitetura ou das ferramentas existentes, as equipes possam consolidar seus logs na plataforma New Relic para análise unificada.

Logs em Contexto

Logs em Contexto (Logs in Context) é uma das funcionalidades mais poderosas da plataforma de observabilidade da New Relic. Ela resolve um dos maiores desafios no troubleshooting de sistemas distribuídos: a necessidade de alternar entre diferentes ferramentas e telas para correlacionar a performance da aplicação com as mensagens de log. Com o Logs in Context, os logs são automaticamente vinculados aos dados de APM, infraestrutura e traces, fornecendo um contexto rico e imediato para a investigação de problemas.

O enriquecimento dos logs com metadados de correlação é a chave para essa funcionalidade. Quando os logs são encaminhados através do agente de infraestrutura da New Relic ou de um agente APM configurado para tal, eles são "decorados" com atributos importantes, como:

- entity.guid e entity.name: identificam a aplicação ou o host que gerou o log.
- trace.id: o identificador do trace distribuído ao qual o log pertence.
- span.id: o identificador do span específico dentro do trace durante o qual o log foi escrito.

Esses atributos transformam uma simples linha de texto de log em um evento de telemetria rico em contexto. Quando a New Relic recebe um log com esses atributos, ela pode automaticamente associá-lo à transação, ao trace e à entidade correspondentes na plataforma.

Os benefícios práticos do Logs in Context são imensos:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Investigação de Erros: na UI de Errors Inbox ou na análise de um erro de transação no APM, é possível ver um botão "See logs" que leva diretamente para as mensagens de log que foram registradas no momento e no contexto daquele erro. Isso elimina a necessidade de buscar manualmente em arquivos de log por timestamps e IDs de requisição.

Análise de Traces Distribuídos: ao analisar um trace distribuído, cada span pode ter logs associados. É possível clicar em um span específico e ver as mensagens de log que foram escritas durante a sua execução, o que é inestimável para entender o que uma aplicação estava fazendo em um determinado ponto no tempo, especialmente para depurar lógica de negócios complexa.

Monitoramento de Infraestrutura: na UI de monitoramento de infraestrutura, ao analisar a performance de um host (CPU, memória, etc.), é possível visualizar os logs gerados por aquele host no mesmo período, ajudando a correlacionar problemas de infraestrutura com eventos de aplicação.

Para habilitar o Logs in Context, é necessário utilizar um método de encaminhamento de logs que suporte o enriquecimento automático (como o agente de infraestrutura) ou configurar a aplicação para adicionar os metadados de correlação manualmente, utilizando as APIs do agente APM. A configuração correta garante que os logs não sejam apenas armazenados, mas se tornem uma parte integrante e acionável da sua estratégia de observabilidade.

Log Patterns (Detecção De Padrões)

Analisar milhões ou bilhões de linhas de log manualmente é uma tarefa impossível. A funcionalidade de Log Patterns da New Relic utiliza algoritmos de Machine Learning para resolver esse problema, agrupando automaticamente mensagens de log semelhantes e identificando tendências e anomalias. Isso permite que as equipes entendam rapidamente o que está acontecendo em seus logs sem a necessidade de ler cada mensagem individualmente.

O processo de detecção de padrões funciona da seguinte maneira: o New Relic analisa o fluxo de logs ingeridos e utiliza técnicas de processamento de linguagem natural e clustering para identificar mensagens que compartilham uma estrutura comum, mas que diferem em seus valores variáveis (como timestamps, IDs, endereços IP, etc.). Por exemplo, as seguintes mensagens de log:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- Login failed for user 'user1' from IP 192.168.1.10 Login failed for user 'user2' from IP 10.0.0.5

Seriam agrupadas no padrão:

- Login failed for user '*' from IP *

Ao agrupar os logs em padrões, a New Relic pode analisar a frequência de cada padrão ao longo do tempo. A UI de Log Patterns exibe os padrões mais comuns, os padrões que tiveram um aumento ou diminuição súbita de frequência (outliers) e os padrões que apareceram recentemente (new patterns). Essa análise de tendências é extremamente valiosa para:

Detecção Proativa de Problemas: um aumento súbito na frequência de um padrão de erro pode indicar um problema sistêmico, mesmo que o erro não seja crítico o suficiente para disparar um alerta individual. A detecção de outliers permite que as equipes investiguem problemas emergentes antes que eles causem um impacto significativo.

Redução de Ruído: ao identificar padrões de log que são muito frequentes e de baixo valor (por exemplo, mensagens de depuração verbosas), as equipes podem tomar decisões informadas sobre quais logs otimizar ou descartar, reduzindo o ruído e os custos de ingestão.

Criação de Alertas Inteligentes: em vez de criar alertas com base em buscas de texto simples, é possível criar alertas com base na frequência de um padrão de log. Por exemplo, um alerta pode ser disparado se a frequência do padrão "Login failed for user '*'" exceder um determinado limiar em um período de tempo, indicando um possível ataque de força bruta.

Otimização de Ingestão: os padrões de log podem ser usados como base para criar regras de exclusão (drop rules). Se um padrão representa logs que são puramente informativos e raramente consultados, uma regra pode ser criada para descartá-los na ingestão, otimizando os custos.

Em suma, a funcionalidade de Log Patterns transforma o mar de dados de log em insights acionáveis, permitindo que as equipes foquem sua atenção nos eventos que realmente importam.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Parsing e Enriquecimento de Logs

Muitos logs são gerados em um formato de texto não estruturado, o que dificulta a consulta e a análise. O parsing de logs é o processo de converter esse texto não estruturado em um formato estruturado de pares chave-valor (atributos). Isso torna os dados de log muito mais fáceis de consultar, filtrar e agregar. O New Relic oferece um sistema de parsing poderoso e flexível que é aplicado durante a ingestão dos logs.

O New Relic já vem com um conjunto de regras de parsing integradas para formatos de log comuns (como logs de acesso do NGINX, Apache, etc.). Quando um log é ingerido com um atributo logtype correspondente, a regra de parsing apropriada é aplicada automaticamente. No entanto, para logs com formatos customizados, é possível criar regras de parsing próprias.

As regras de parsing no New Relic são compostas por duas partes principais:

Critério de Correspondência (Matching Criteria): uma consulta NRQL (usando a cláusula WHERE) que define a quais logs a regra de parsing deve ser aplicada. É uma boa prática usar um atributo como logtype para direcionar as regras de parsing, mas qualquer atributo pode ser usado.

Lógica de Parsing (Parsing Logic): a expressão que define como extrair os dados do campo de log (geralmente o campo message). O New Relic suporta duas linguagens para a lógica de parsing:

- Grok: um conjunto de padrões pré-definidos que representam expressões regulares comuns. Grok é a maneira mais fácil e legível de fazer parsing de formatos de log padrão. Por exemplo, %{IP:client_ip} extrai um endereço IP para o atributo client_ip.
- Expressões Regulares (Regex): para formatos de log mais complexos ou não padrão, é possível usar expressões regulares (na sintaxe Java) com grupos de captura nomeados para extrair os atributos.

As regras de parsing são aplicadas na pipeline de ingestão, antes que os dados sejam escritos no NRDB e antes de qualquer outro enriquecimento de dados. É importante notar que apenas a primeira regra que corresponder e for aplicada com sucesso a um log será utilizada. Se múltiplas regras corresponderem, uma será escolhida de forma não determinística.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O enriquecimento de logs é o processo de adicionar mais atributos contextuais a um log após o parsing. Isso pode incluir:

- GeoIP: o New Relic pode enriquecer automaticamente os logs com informações de geolocalização (cidade, país, etc.) com base em um atributo de endereço IP.
- Atributos de Entidade: como visto no Logs in Context, o agente de infraestrutura adiciona atributos sobre o host que gerou o log.
- Atributos de Trace: o agente APM adiciona atributos de trace.id e span.id.

O parsing e o enriquecimento transformam logs brutos em dados estruturados e ricos em contexto, que podem ser facilmente explorados na UI, consultados com NRQL para criar dashboards e alertas, e correlacionados com o restante da telemetria do sistema.

Filtrar e Reduzir Ingestão de Logs

Embora a centralização de logs seja poderosa, nem todos os logs têm o mesmo valor. Aplicações em modo de depuração, sistemas verbosos ou bots podem gerar um volume enorme de logs de baixo valor que aumentam os custos de ingestão e armazenamento e criam ruído, dificultando a localização de informações importantes. Para gerenciar isso, o New Relic oferece a capacidade de filtrar e descartar (drop) logs indesejados diretamente na pipeline de ingestão.

A principal ferramenta para isso são as Drop Rules. Uma drop rule é uma regra, definida por uma consulta NRQL, que especifica quais logs devem ser descartados antes mesmo de serem escritos no banco de dados da New Relic (NRDB). Como os logs são descartados na ingestão, eles não consomem armazenamento, não são faturados e não aparecem nos resultados de consulta.

Criar uma drop rule é um processo simples na UI do New Relic. O usuário define uma consulta NRQL que corresponde exatamente aos logs que deseja descartar. Por exemplo, para descartar todos os logs de depuração de uma aplicação específica, a consulta poderia ser:

```
logtype = 'my-app' AND level = 'DEBUG'
```

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Uma vez que a regra é ativada, todos os logs futuros que corresponderem a essa consulta serão descartados.

A decisão sobre quais logs descartar é estratégica e depende das necessidades de cada equipe. Algumas diretrizes comuns incluem:

Descartar Logs de Nível de Depuração (Debug) em Produção: logs de depuração são essenciais durante o desenvolvimento, mas geralmente não são necessários em produção e podem gerar um volume massivo de dados.

Filtrar Requisições de Health Check: muitos sistemas de orquestração e balanceadores de carga fazem requisições de health check constantes. Os logs de acesso para essas requisições geralmente não são úteis para análise e podem ser descartados.

Excluir Logs de Bots e Crawlers: logs de acesso gerados por bots de busca e outros crawlers podem poluir os dados de tráfego real do usuário.

Remover Logs Repetitivos e de Baixo Valor: a funcionalidade de Log Patterns pode ajudar a identificar padrões de log que ocorrem com muita frequência, mas que não fornecem informações acionáveis. Esses são candidatos ideais para drop rules.

É importante usar as drop rules com cuidado, pois os dados descartados não podem ser recuperados. Antes de criar uma regra, é uma boa prática analisar o uso dos logs em questão. Se uma categoria de logs nunca é consultada ou usada para alertas, é provável que seja seguro descartá-la.

Ao combinar o parsing (para estruturar os dados), o Log Patterns (para identificar candidatos) e as drop rules (para filtrar), as equipes podem otimizar sua ingestão de logs, reduzindo custos e focando sua atenção nos dados que realmente importam para a operação e a segurança de seus sistemas.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Observabilidade Unificada

Quatro Pilares Melt

A observabilidade eficaz de sistemas complexos e distribuídos requer mais do que apenas uma única fonte de dados. É necessária uma visão holística que combine diferentes tipos de telemetria para fornecer uma compreensão completa do comportamento do sistema. A New Relic baseia sua plataforma de observabilidade em quatro pilares fundamentais de dados de telemetria, conhecidos pelo acrônimo MELT: Metrics, Events, Logs, and Traces.

Metrics (Métricas): métricas são agregações numéricas de dados medidos ao longo do tempo. Elas são altamente eficientes para armazenar e consultar, tornando-as ideais para dashboards e alertas sobre a saúde geral do sistema. Exemplos de métricas incluem a utilização da CPU de um host, a taxa de transferência (throughput) de uma aplicação, a taxa de erro e o tempo de resposta médio. As métricas nos dizem "o quê" está acontecendo em um sistema (por exemplo, "a CPU está em 95%").

Events (Eventos): eventos representam ocorrências discretas e imutáveis que acontecem em um ponto específico no tempo. Cada evento é um registro individual com um conjunto de atributos que descrevem o que aconteceu. Exemplos incluem um evento Transaction que representa uma única requisição web, um evento PageView que representa a carga de uma página no navegador de um usuário, ou um evento customizado que representa uma ação de negócio, como ShoppingCartPurchase. Os eventos nos dão o contexto detalhado por trás das métricas e nos permitem fazer perguntas arbitrárias sobre o nosso sistema (por exemplo, "quantos usuários do Brasil viram um erro 500 na última hora?").

Logs (Registros): logs são registros textuais, com timestamp, da atividade do sistema. Como vimos, eles podem ser estruturados ou não estruturados e fornecem o nível mais granular de detalhe sobre o que uma aplicação estava fazendo em um determinado momento. Os logs são a "voz" da aplicação e são indispensáveis para a depuração e a análise forense. Eles nos dizem "por quê" algo aconteceu (por exemplo, a exceção exata que causou uma falha).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Traces (Rastreamentos): traces, como detalhado anteriormente, são representações do fluxo de uma requisição através de múltiplos serviços em uma arquitetura distribuída. Eles são compostos por spans e mostram a jornada completa de uma operação, de ponta a ponta. Os traces nos dizem "onde" um problema está ocorrendo em um sistema distribuído, identificando qual serviço ou dependência está causando latência ou erros.

A verdadeira magia da plataforma New Relic reside na sua capacidade de unificar e correlacionar esses quatro tipos de dados. Ao ingerir todos os dados MELT em um banco de dados de telemetria unificado (NRDB), a New Relic permite que as equipes naveguem de forma transparente entre eles. Um pico em uma métrica de latência pode ser investigado analisando os eventos de transação correspondentes, que por sua vez podem ser detalhados em um trace distribuído para identificar o serviço problemático, e finalmente, os logs daquele serviço podem ser examinados para encontrar a causa raiz do erro. Essa capacidade de correlação é o que define a verdadeira observabilidade unificada.

Logs em Contexto com Outros Dados

A funcionalidade de Logs em Contexto é um exemplo primordial da filosofia MELT em ação, demonstrando o poder de correlacionar diferentes tipos de dados de telemetria. Ao tratar os logs não como um silo de dados isolado, mas como uma parte integrante do ecossistema de observabilidade, a New Relic permite que as equipes de engenharia obtenham insights muito mais profundos e acelerem drasticamente a solução de problemas.

A correlação de logs com outros dados de telemetria, como métricas de APM e dados de infraestrutura, fornece uma narrativa completa do comportamento do sistema. Vejamos alguns cenários práticos onde essa correlação é inestimável:

Correlação com Dados de APM: imagine que um alerta de APM é disparado para uma alta taxa de erro em uma transação específica. Na UI de APM da New Relic, o engenheiro pode ver o detalhe do erro, incluindo o stack trace. Com o Logs em Contexto, haverá um link direto para visualizar os logs que foram gerados pela aplicação no exato momento em que aquele erro ocorreu. Isso pode revelar informações cruciais que não estão no stack trace, como os valores de variáveis de entrada, o estado de uma conexão externa ou uma mensagem de erro mais detalhada

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

de uma biblioteca de terceiros. A navegação é bidirecional: a partir de uma mensagem de log na UI de Logs, se ela estiver enriquecida com um trace.id, é possível saltar diretamente para o trace distribuído correspondente.

Correlação com Dados de Infraestrutura: suponha que o monitoramento de infraestrutura mostre um pico de utilização de CPU em um determinado host. Sem contexto, pode ser difícil saber o que causou esse pico. Com o Logs em Contexto, o engenheiro pode, na mesma tela, visualizar os logs gerados por todas as aplicações rodando naquele host durante o período do pico. Isso pode revelar, por exemplo, que uma aplicação começou a logar um grande volume de erros, ou que um processo em batch foi iniciado, explicando o consumo de CPU. Essa correlação direta entre a camada de aplicação (logs) e a camada de infraestrutura (métricas de host) é fundamental para uma análise de causa raiz completa.

Correlação com a Experiência do Usuário Final: os dados de logs também podem ser correlacionados com dados de monitoramento de Browser e Mobile. Por exemplo, um erro de JavaScript reportado pelo agente do Browser pode ser correlacionado, através de um trace.id propagado da chamada de API do frontend para o backend, com os logs do serviço de backend que atendeu àquela chamada. Isso permite que a equipe de frontend e a equipe de backend colaborem de forma mais eficaz, entendendo o impacto completo de um problema, desde a experiência do usuário até a implementação do serviço.

Em resumo, o tratamento de logs como "registros da atividade do sistema" que podem ser vinculados a todos os outros sinais de telemetria é o que eleva o gerenciamento de logs de uma simples ferramenta de busca para um componente estratégico da observabilidade unificada. A capacidade de ver logs ao lado de dados de APM, infraestrutura, traces e experiência do usuário final elimina as suposições e permite que as equipes resolvam problemas de forma mais rápida e baseada em dados.

Trace Data e Spans

Os dados de trace e seus componentes fundamentais, os spans, são o pilar da observabilidade em arquiteturas distribuídas e um elemento central do framework MELT. Cada trace representa a jornada completa de uma única requisição através de múltiplos serviços, e cada span representa uma unidade de trabalho específica dentro

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

dessa jornada. A análise detalhada desses dados é o que permite diagnosticar gargalos de performance e erros em sistemas complexos.

Cada span é, em essência, um objeto de dados rico em informações, contendo um conjunto de atributos padrão e customizáveis que descrevem a operação que ele representa. Alguns dos atributos mais importantes de um span incluem:

- trace.id: o identificador único que agrupa todos os spans pertencentes ao mesmo trace.
- span.id: o identificador único do próprio span.
- parent.id: o identificador do span pai, que estabelece a relação hierárquica dentro do trace.
- name: um nome legível para o span, como o nome de um método, uma URL de requisição HTTP ou uma consulta de banco de dados.
- duration: a duração da operação representada pelo span, em milissegundos.
- timestamp: o momento em que o span iniciou.
- service.name: o nome do serviço que gerou o span.
- host: o host onde o serviço estava rodando.
- Atributos de Erro: atributos que indicam se o span terminou com um erro.
- Atributos Específicos: atributos que fornecem contexto adicional, como http.method e http.status_code para chamadas externas, ou db.statement para chamadas de banco de dados.

Todos esses spans são enviados para a New Relic e armazenados como eventos do tipo Span no NRDB. Isso significa que eles podem ser consultados e analisados em profundidade utilizando a New Relic Query Language (NRQL). As consultas NRQL sobre dados de Span permitem responder a perguntas complexas sobre a performance do sistema, como:

```
SELECT average(duration) FROM Span WHERE service.name =
'OrderService' AND name = 'External/PaymentGateway/charge'
FACET http.status_code
```

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Esta consulta calcula a duração média das chamadas para um gateway de pagamento, facetada pelo código de status HTTP de resposta, permitindo identificar se determinados erros estão associados a uma maior latência.

```
SELECT count(*) FROM Span WHERE service.name =
'InventoryService' AND error.message IS NOT NULL FACET name
```

Esta consulta conta o número de spans com erro no serviço de inventário, agrupados pelo nome da operação, ajudando a identificar quais operações são mais propensas a falhas.

Além da análise via NRQL, a UI de Distributed Tracing da New Relic, como visto anteriormente, fornece visualizações intuitivas (Timeline, Latency, Waterfall) que facilitam a identificação de latências, chamadas externas lentas e a propagação de erros ao longo de um trace.

A correlação de traces com os outros pilares do MELT é o que completa o ciclo de observabilidade. Um span em um trace pode ser correlacionado com os logs gerados durante sua execução (Logs in Context). A agregação de milhares de spans pode gerar métricas de performance (por exemplo, a duração média de um tipo de span). E cada trace em si pode ser considerado um evento complexo, que pode ser analisado em conjunto com outros eventos de negócio. Essa interconexão de dados é o que permite que as equipes passem de uma visão macro da saúde do sistema (métricas) para a análise de uma única requisição problemática (trace) e, finalmente, para o detalhe do código que a executou (logs).

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Esta aula apresentou os fundamentos do **Distributed Tracing** em arquiteturas distribuídas, detalhando a estrutura de traces e spans (entry, exit e in-process) dentro do ecossistema New Relic. Exploramos como a propagação de contexto via **W3C Trace Context** garante a correlação entre serviços síncronos e assíncronos, permitindo uma visão clara do fluxo de dados. Além disso, discutimos estratégias de amostragem, diferenciando abordagens *head-based* e *tail-based*, com foco especial no poder do **Infinite Tracing** para capturar anomalias críticas.

Para a análise prática, o conteúdo abordou ferramentas de visualização como as visões de Timeline e Waterfall, além do uso de **NRQL** para consultas personalizadas de performance. Complementando o rastreamento, introduzimos o **Log Management**, focando em métodos eficientes de coleta, encaminhamento e a importância dos **Logs in Context** para conectar registros de eventos diretamente a traces e erros, acelerando o diagnóstico de falhas.

Por fim, a aula demonstrou técnicas avançadas de gerenciamento de dados, como a detecção automática de **Log Patterns** e o enriquecimento de logs via **Grok e Regex**. Também tratamos da otimização de custos e performance através de **Drop Rules** para controle de ingestão. Tudo isso convergiu para o modelo **MELT**, consolidando uma visão de observabilidade unificada que integra métricas, eventos, logs e traces em um único plano de análise.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

REFERÊNCIAS

NEW RELIC. **APM logs in context**. Disponível em: https://docs.newrelic.com/docs/logs/logs-context/get-started-logs-context/. Acesso em: 28 jan. 2026.

NEW RELIC. **Discover value in log data with patterns**. Disponível em: https://docs.newrelic.com/docs/logs/ui-data/find-unusual-logs-log-patterns/. Acesso em: 28 jan. 2026.

NEW RELIC. **Drop data using NerdGraph**. Disponível em: https://docs.newrelic.com/docs/data-apis/manage-data/drop-data-using-nerdgraph/. Acesso em: 28 jan. 2026.

NEW RELIC. **Get started with log management**. Disponível em: https://docs.newrelic.com/docs/logs/get-started/get-started-log-management/. Acesso em: 28 jan. 2026.

NEW RELIC. **Infinite Tracing overview.** Disponível em: https://docs.newrelic.com/docs/distributed-tracing/infinite-tracing/introduction-infinite-tracing/. Acesso em: 28 jan. 2026.

NEW RELIC. **Introduction to distributed tracing**. Disponível em: https://docs.newrelic.com/docs/distributed-tracing/concepts/introduction-distributed-tracing/. Acesso em: 28 jan. 2026.

NEW RELIC. **Parsing log data.** Disponível em: https://docs.newrelic.com/docs/logs/ui-data/parsing/. Acesso em: 28 jan. 2026.

NEW RELIC. **Technical distributed tracing details.** Disponível em: https://docs.newrelic.com/docs/distributed-tracing/concepts/how-new-relic-distributed-tracing-works/. Acesso em: 28 jan. 2026.

NEW RELIC. **Understand and use the distributed tracing UI**. Disponível em: https://docs.newrelic.com/docs/distributed-tracing/ui-data/understand-use-distributed-tracing-ui/. Acesso em: 28 jan. 2026.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

PALAVRAS-CHAVE

Distributed Tracing, Logs, Observabilidade

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com